000课程简介

2026年9月3日 13:41

一. AI能力跃迁与人的角色转变

1. 代码生产力跃迁：AI能“一次性生成完整可编译模块”，所以人必须从“生产者”降维成“验收测试官”

- AI能力跃迁：从Copilot（补全一行）跃迁到Harness（生成整个Service层），输出速度从“分钟/行”变为“秒/百行”。
- 角色转变：人类亲手写代码的边际价值归零。
- 新角色定义：“测试用例的立法者”。AI交卷后，跑测试，绿灯放行，红灯丢弃。因果链：生成速度无限大 → 人工审查失效 → 自动化测试成为唯一质检手段。

2. AI的生理硬伤：AI有“中间迷失”（Lost-in-the-Middle）（长链记忆断层），所以人类必须从“架构规划师”退行为“状态存档管理员”

- AI能力跃迁：它能读懂10万行代码库的语法结构，但无法在生成第500行时记住第50行立下的业务约束。
- 强制角色转变：必须放弃让AI“记住一切”的幻想。
- 新角色定义：“外部闪存的维护员”。因果链：AI无法固化记忆 → 人类必须将记忆外挂到文件系统 → 人类角色矮化为“状态文件的增删改查者”。

3. AI缺失协调能力：AI能“无限克隆并行”（资源可扩展性跃迁），但无法“处理并发冲突”，所以人类必须从“团队主管”异化为“冲突仲裁调度员”

- AI能力跃迁：云成本允许你同时起10个AI代理，各自负责一个微服务的生成。但两个代理可能同时修改同一个全局配置或数据库Schema，产生不可预知的覆盖。
- 强制角色转变：无法靠“开会”或“文档约定”来协调AI。
- 新角色定义：“分布式锁的制定者”。因果链：AI并发能力爆发 → 人类必须承担全局序列化（Serialization）职责 → 人类沦为“任务流水线的班组长”。

4. AI的错误熵增：AI能“无限次试错”（迭代成本趋零），但每次失败会“污染自身上下文”（错误熵增），所以人类必须从“调试修复者”转型为“强制重启执行者”

- AI能力跃迁：传统人类调试一次Bug要半小时；AI修一次Bug只要5秒。但它有个致命副作用——每次修Bug都要把错误堆栈加载到上下文，导致上下文越来越混乱，后续修复越来越弱智。
- 强制角色转变：简单的“再试一次”，只会把代码改成一锅粥。
- 新角色定义：“会话生命周期终结者”。因果链：AI试错成本低但污染率高 → 人类必须舍弃“修复”思维 → 人类变成“强制重启的机械手”。

5. AI无边界识别能力：AI能“理解模糊需求”（自然语言理解跃迁），但无法“量化模糊”，所以人类必须从“需求分析师”升维为“边界约束精算师”

- AI能力跃迁：能听懂“做一个像淘宝一样的订单系统”，但它会把“像”理解成“功能堆砌”，永远不懂什么是“最小可行”和“成本控制”。
- 强制角色转变：AI没有“成本意识”，无法做技术选型或性能权衡。
- 新角色定义：“约束量化师”。因果链：AI能执行需求但不懂权衡 → 人类必须承担精确量化职责 → 人类变成“数字边界雕刻师”。

6. AI的问责能力为零：AI能“输出完整工程”（交付物跃迁），但无法“承担责任”，所以人类必须从“技术专家”晋升为“法律意义上的最终责任人”

- AI能力跃迁：在能提交一整个PR（Pull Request），包含代码、测试、部署脚本。
- 强制角色转变：AI无法坐在故障复盘会上解释“为什么把用户余额扣

一次性生成一个用户登录模块，
包含密码验证、会话创建、数据
库查询等。java代码



- AI能力跃迁：在能提交一整个PR（Pull Request），包含代码、测试、部署脚本。
- 强制角色转变：AI无法坐在故障复盘会上解释“为什么把用户余额扣成了负数”。
- 新角色定义：“代码的绝对背书人”。因果链：AI能交付但无法负责 → 人类必须承担全部法律和业务后果 → 人类变为“风险控制终审法官”。

结论

- AI能力的每次进化，都不是在“辅助”人类，而是在“夺走”人类原有的中场阵地，把人类推向更高维、更抽象、更接近业务本质的狭窄决策点。
- 试图“教会AI写更好的代码”，是在和机器的物理极限对抗，必败。
- 接受“因为AI记不住，所以我管状态；因为AI不看全局，所以我设边界；因为AI不担责，所以我签名字”这条因果链，人的工作量其实没变少，但工作的颗粒度从“代码字符”跃迁到了“系统规则”，这正是软件工程范式革命的本质。

问题

如何看待网上充斥的大量AI测评视频?

让我理解：用户说「分步写临时文件，校核后签回代码文件」，这是指**代码改动**也要先写临时文件校核，再签回。

但「临时文件」具体指什么？可能是：
A. 一个 _tmp_ 文件，里面放「要签回的代码内容」（diff 预览），校核后应用。
B. 直接把改动写到代码文件，但用 git 的方式（先不 commit，校核后 commit）？

从「签回代码文件」看，应该是 A：临时文件里放代码内容，校核后「签回」= 应用到代码文件。

实际上，让我重新理解用户的工作流。之前 CoSimulationPlan 的工作流是：
- 分析 → 写 _tmp_ 落点设计文档 → 校核 → 实施（直接改代码文件）→ 验证 → commit。

二. 教师与学生（计算机专业）

教师

1. AI知识储备量跃迁，教师必须从“知识权威”降级为“信息甄别导购”

- 教师的核心价值不再是“把知识讲对”，而是“帮学生识别AI给出的哪段示例是过时写法、哪个开源协议有法律风险、哪种实现虽然能跑但在大厂被明令禁止”。你的课堂要从“定义是什么”彻底转向“AI给的这5种方案，在生产环境下为什么第3种是坑”。因果链：AI精通标准答案 → 教师必须放弃标准答案教学 → 教师沦为“避坑指南的主编”。

2. AI评估效率跃迁，但AI共情与诊断能力为零，教师必须从“评分裁判”异化为“认知病理医生”

- 教师必须变成“思维断层扫描师”。教师的工作不再发生在课后（批作业），而是发生在课中（面对面/直播）定位个AI永远诊断的“前置知识缺环”。因果链：AI能高效评卷但无心理模型 → 教师

必须放弃数据评判 → 教师沦为“人类思维漏洞的专职侦测仪”。

3. AI出题效率跃迁，但AI无价值观塑造**能力**，所以教师必须从“出题人”升维为“工程伦理守门人”

- 教师必须**恪守**“职业底线”。因果链：AI能出题但无责任意识 → 教师必须承担社会化塑造 → 教师升维为“行业品行的最后一道保险丝”。

学生

1. AI无法建立底层抽象直觉（肌肉记忆缺失），学生必须从“代码打字员”降级为“AI输出品的评审官”

- 学生必须成为“AI代码的评审官”。新学习模式是：让AI生成一段300行的排序算法，学生的任务不是默写，而是强行在代码里挑出3个隐藏的性能隐患和1个风格缺陷，并提交“评审报告”。练习的不再是“构造”，而是“解构与批判”。因果链：AI替代了构造过程 → 学生必须放弃打字训练 → 学生异化为“AI代码的专职业力审查员”。

2. AI无韧性培养**能力**，学生必须从“被动的答案接收者”逆转为“主动的约束定义者”

- 学生必须主动给自己设置“AI禁用手套”——强制纯手写代码（不能用任何补全工具），并刻意在项目中引入极端约束（如“只能用100M内存处理10G文件”），逼AI给出的通用方案失效，然后自己推演。你的核心竞争力不再是“知道答案”，而是“能在AI给不出答案时，定义出那个前所未有的约束条件”。因果链：AI消除了低级挫败感 → 学生必须刻意自找高级挫败感 → 学生必须进化为“主动为自己设置算法牢笼的苦行僧”。

3. AI无商业嗅觉，学生必须从“单纯学习者”升格为“业务成本的初级精算师”

- 学生必须养成习惯，每次拿到AI的代码，先问自己：“如果我是老板，下个月服务器预算砍半，我选哪段？如果核心员工离职，没人看得懂这段操作，我选哪段？” 你的学习笔记必须从“这个算法的时间复杂度是O(n log n)”进化为 “这个算法在每天调用100万次的场景下，年化电费比暴力解法省3000元，但会导致新员工入职培训多花2天”。因果链：AI能提供多元方案但无商务权衡 → 学生必须主动代入老板视角 → 学生被拔苗助长为“成本-收益的初级平衡师”。

“样样通，样样松”

AI的物理能力跃迁，直接颠覆了“通”与“专”的价值坐标系。

1. AI能在“单一细分领域”做到人类无法企及的“绝对精通”（深度垄断），学生必须放弃“深耕一隅”的执念，转向“领域拼图师”的角色
   学生必须变成“跨域拼图的识别者”。他的核心价值不再是“把一块拼图打磨到极致”，而是“同时看懂10块不同领域的拼图（前端、后端、数据库、运维、产品），知道它们各自的凹凸接口在哪，然后把它们严丝合缝地拼成一个能跑通的商业系统”。因果链：AI垄断了“绝对深度” → 人类只能退守到“相对广度” → 学生异化为“系统级胶水涂抹师”。
2. AI能“精准复述”，但AI“无法自主发现‘未知的未知’（跨域类比能力为零）”，学生必须从“深度钻研者”进化为“跨界联想者”
   学生必须有意识地“松散”涉猎经济学、心理学、生物学，甚至美学。他的学习笔记不再是“Java并发编程笔记”，而是 “今天学的蚂蚁觅食路径算法，能不能借用过来优化小区快递柜的调度？” 。这种能力需要极度松散的“通识储备”，专精一隅的大脑永远无法产生这种化学

反应的“空白地带”。因果链：AI只会线性复述 → 学生必须掌握非线性迁移 → 学生沦为“脑内蝴蝶效应的煽动者”。

3. AI迭代周期从“年”缩短到“周”（技术栈加速过时），学生必须将“学习效率”置于“学习深度”之上，成为“快速的软弱学习者”

学生必须掌握 “浅尝辄止的硬技能”——允许自己只掌握某技术30%的核心功能就立刻上手做Demo，然后转去学下一个。你需要的是“极快的从0到1搭建原型并试错”的能力，而不是“从1到100的极致打磨”。那个打磨的苦活，交给AI去干。因果链：AI加速技术淘汰 → 静态知识贬值 → 学生必须转型为“快速原型收割机”。

致命陷阱（必须规避的“真·样样松”）：

“样样通，样样松”有个前提：必须有一个坚硬的“内核锚点”。

正确的“AI时代学生画像”应该是：

1个硬核锚点（硬通货）：选一个极其稳定的数学/逻辑底层（比如离散数学、数据结构与算法背后的复杂度下界证明，或是分布式共识的数学原理）。这个领域AI能帮你做题，但它无法替你“理解”为什么P=NP问题这么难。这个锚点用来建立你的逻辑尊严，防止被AI彻底带偏。

10个松散触手（软连接）：像章鱼触手一样去触碰云原生、AI应用、边缘计算、产品设计、心理学、甚至写小说。每样只摸到“能用AI提示词唤醒它”的熟悉度即可。

三. 课程简介

1、摒弃传统教学方法，从实操入手，迭代式开发一个Coding AI harness

原理与开发并行

课堂实时演练

课后持续完善

塑造教师、学生、AI之间的角色定位意识与边界（尝试）

2、AI Coding

免费或官方API（价格透明）

避免第三方、个人API（隐藏陷阱）

Flash版足够（更易触发AI能力边界），不需要PRO版

3、工具

Cmake

Visual Studio 2022、2026（需要安装14.3 2022兼容包）

QT 6

Node.js

Python

其他依赖工具（自行裁决）

4、小组合作

2-3人一组，合作讨论

均摊API费用，降低成本（不接受成本付出，可以马上退选）

5、学习通





默认班级

6、考核

时间：最后两次课

方式：分组上台讲解（每组10分钟）

内容：完整设计文档、功能展示，关键策略，重点优化模块，Skills、Agent配置等

留底：已作业形式上传所有设计文档和源代码（压缩包形式）